

Rogue's Oath — Design Doc (vo.3)

Part 1 — The Game

You and a couple of friends descend into a procedurally generated dungeon. It's a traditional roguelike at heart — permadeath, tile-based tactics, monsters, loot, chokepoints, "do we fight this or run" — but everyone is playing at once, live, in the same dungeon. Nobody takes turns. Nobody waits.

Combat is slow enough to think. An ogre winding up to hit you is telegraphed seconds in advance — enough time to decide, not enough time to do everything. You can attack, or you can drink the potion. Not both. And once you choose, you're locked in: no canceling, no dodge-rolling out of it, no take-backs. The game never tests your reflexes — it tests whether you made the right call. It's the kind of combat you can play with a sandwich in one hand, but a wrong decision still gets you killed.

Runs are short — winnable in a sitting, under an hour or so. Death is permanent. The dungeon is different every time. Builds are simple enough that you can glance at a friend's character and know what they do.

Presentation: 2D top-down tiles (decided vo.3). Sprites on a tile grid, in the DCSS / Rogue Fable visual lineage.

Feel targets: the deliberation of DCSS, the pacing of Neverwinter Nights (pacing only — the presentation is 2D), the streamlined onboarding of Rogue Fable, playable with friends over a network without anyone's ping mattering.

Explicitly not: an action game, a twitch game, an MMO, a turn-based game with a lobby.

Part 2 — System Spec

2.1 The Commitment Rule (core invariant)

1. Every action — attack, cast, heal, item use, movement — has a duration and plays out to completion once started.
2. No action can be canceled, interrupted by player input, or redirected after commit.
3. There is no active dodge, block, or escape input. Defense is stats, positioning, and choosing well before committing.
4. Design test for every future mechanic: does it let a player back out of a decision for free? If yes, redesign it.

2.2 Movement

1. World is a tile grid. Every entity occupies exactly one tile.
2. Moving = a glide: entity commits to an adjacent tile, then smoothly animates into it over a real-time duration. Position data is discrete; presentation is smooth.
3. Glide duration is stat-driven, in discrete speed tiers (breakpoints), not a continuous scale.
4. A glide obeys the Commitment Rule: once started, it finishes. Being hit does not interrupt it.
5. Tile reservation: on commit, the destination tile is reserved for the glide's duration. A second entity committing into a reserved tile fails (input rejected; no queuing for v1).
6. Attack of opportunity: starting a glide out of a tile adjacent to a hostile that is alive and able to act grants that hostile one free attack.
7. Open: diagonal movement (allowed? same cost?). Affects LoS and corner math — decide before dungeon-gen work. (*Elevated to Open Question 3 — this blocks implementation.*)
8. **Commit feedback (added v0.3).** The feedback rule (2.3.4) applies to movement: pressing a move renders an instant, local "**commit sent**" acknowledgment, so the player always knows the input registered. The **verdict** — glide start, or a rejection "bonk" (sound + visual) when the destination is reserved (2.2.5) — always comes from the server. The client never predicts the outcome, in either direction; there is no client-side authority anywhere. A rejected move must never be confusable with dropped input, and a locally-guessed rejection must never contradict a server accept.

2.3 Combat Resolution

1. Two-step resolution per attack: (a) accuracy vs. evasion/resist → hit or miss; (b) if hit, roll damage magnitude.
2. All hit/evade/resist chances are passive, build-derived stats. Rolled silently, no player input involved.
3. Roll types (tentative): miss, crit, block, passive dodge, spell resist.
4. Every distinct roll outcome has a distinct, unambiguous feedback signal (sound + visual + combat log line). A player must never confuse "the roll failed" with "my input didn't register." (*This rule extends to movement rejection — see 2.2.8.*)
5. Combat presentation stays abstracted (targeted commands, no implied precise physical contact). Do not show pre-commit hit percentages.
6. RNG budget: keep output randomness (roll swing) low-magnitude. Replayability comes from input randomness — dungeon gen, loot, encounters — not from swingy rolls. A bad roll should always leave the player a real next decision, never just erase a correct one.

2.4 Periodic Effects (DoTs / HoTs / regen / buffs)

1. No shared global tick. Every periodic effect runs its own independent timer at whatever cadence suits it (a 1-dmg-per-second poison and a big heal-every-30s regen coexist freely).
2. Deferred: a deliberately loud, group-visible "shared beat" for coordination moments — only if a concrete need appears.

2.5 Multiplayer Architecture

1. Engine: Godot, built-in high-level multiplayer (ENet, MultiplayerSpawner/Synchronizer, RPCs). Server-authoritative.

2. Reuse (made precise in v0.3): plumbing comes from the Magick With Friends

`framework/` **layer** (the matured successor to the Friend Slop Framework). Scope of reuse:

- **Lift near-verbatim:** `framework/autoloads/network_manager.gd` (swappable transport contract — all ENet code isolated behind `host_game` / `join_game` / `disconnect_game` / `kick_peer`; verified to contain no tick-rate or state-sync assumptions), the `GameManager` config / player-name pattern, the main-menu host/join UI, and the dev console (`framework/ui/console/` — host-gated commands, invaluable for multiplayer testing).
 - **Lift the patterns, rewrite the code:** session flow from MWF `main.gd` — the `peer_ready` RPC with duplicate-spawn guard, capacity spawn-gate with kick backstop, host-left handling, peer-disconnect cleanup. (The host-left pattern is client-side UX for the disconnect *moment* — freeze, overlay, return to menu — and is needed under any host-disconnect policy; lifting it does not pre-answer Open Question 2.)
 - **Explicitly excluded:** `player_input_synchronizer_component.gd` and `remote_visual_smoother.gd`. Both stream continuous per-frame state — the opposite of the event-based commit model below. All movement/action networking is written from scratch for this game.
3. All gameplay sync is event-based, not position-streamed: replicate discrete commits (`glide_to(tile)`, `attack(target)`, `use_item(id)`), each stamped with duration/outcome by the server. The commitment model makes this natural — entities are always either idle or inside a known, finite action.
 4. No client-side prediction needed for v1 — round-trip latency is absorbed by the slow, telegraphed pacing by design.
 5. Target scale: small friend groups (2–6 players). No matchmaking/lobby service for v1. All players join before the run starts; there is no mid-run join (see 2.7).

2.6 UI/HUD (direction only — build later)

1. Aesthetic: simple and readable 2D, in the DCSS / Rogue Fable lineage (NWN informs the HUD layout conventions below, not the rendering).
2. Multiplayer additions a solo roguelike HUD lacks, all v1-relevant:

- persistent party status (portraits + HP, NWN-style)
- a visible tell on any player currently locked in a commit (so teammates read each other's state)
- nameplates / ally-enemy color coding
- minimap that also shows teammate positions
- combat log carrying the per-roll feedback from 2.3.4

3. Not needed ever: initiative / turn-order UI. There are no turns.

2.7 Explicitly Out of Scope for v1

- Matchmaking, lobbies, NAT traversal services
- Client-side prediction / rollback netcode
- Shared-beat mechanic (2.4.2)
- Final visual style
- Character build systems beyond placeholder stats (design pass needed — Rogue Fable-style legibility is the bar: a build should be readable from a handful of numbers)
- Dungeon generation design (needs its own pass; it is the game's primary randomness source, so it deserves one)
- Mid-run join / late-join state snapshot (added v0.3): everyone starts the run together. This also removes the "what does a late joiner see mid-glide" replication problem from v1 entirely.

Part 3 — Appendix: Why (short version)

Why commitment instead of turns? What makes a roguelike turn tactical isn't the pause — it's that the decision is spent once made. Cooldown-based real-time games (EQ/WoW) gate how often you act, but let you cancel and reposition freely, which is why they feel loose. Hard commitment recreates the weight of a turn with zero waiting, which is what lets this stay real-time and multiplayer.

Why no reflex demands? Dark Souls proves commitment works in real time, but couples it to twitch execution. Decoupling them (slow telegraphs + hard commits) keeps the judgment test and drops the execution test — and buys near-total lag tolerance for free.

Why grid + glide? Grid gives clean tactics math (adjacency, chokepoints, AoO triggers). Glide keeps it from reading as turn-based. Tibia has shipped exactly this in live multiplayer for 25+ years; the speed-tier quantization follows Diablo's frame-breakpoint precedent.

Why lock movement too? Free repositioning is an escape hatch with a different name — leave it uncommitted and optimal play collapses into perpetual kiting instead of hard choices.

Why passive rolls but no active dodge? Passive evasion is a build-time decision resolved silently — it never un-makes a real-time choice. An active dodge button does. One is texture, the other is an escape valve.

Why no global tick? EQ's 6-second tick was 1990s MUD server economics, not a design ideal.

Independent per-effect timers express any cadence a designer wants; a shared clock can't.

Why low output-RNG? More decision complexity needs less injected randomness to stay fresh (the MTG-vs-Hearthstone principle). This game is decision-dense and its choices are irrevocable — big post-commit roll swings would fight the core pillar.

Why event-sync networking? The design only ever changes state through discrete commits, so replicating events is both cheaper and truer to the model than streaming positions.

Part 4 — Open Questions (for Jeff)

*No decisions here — each item frames a tradeoff to discuss. Items marked **[BLOCKS IMPLEMENTATION]** need answers before the affected system gets built; the rest can wait.*

1. **Death mid-run.** The biggest unaddressed design question. Permadeath + co-op + ~1-hour runs means a dead player may spectate for most of a session. Candidate directions: pure spectate (purest permadeath, simplest, boring for the dead), a ghost with minor utility (scouting, small buffs — engaged without undoing permadeath), or a costly revive (softens permadeath — touches the core pillar). Related sub-question the Commitment Rule forces: if a player dies mid-commit (killed by an AoO mid-swing, glides onto a trap), does the committed action still complete post-mortem?
 2. **Host disconnect policy.** Server-authoritative with a friend hosting means a host crash can erase a 50-minute run. Accept that consciously (and add it to 2.7), or define a save-on-quit / resume story? Note the reused host-left UX (2.5.2) handles the disconnect moment either way — this question is about whether the run state survives it.
 3. **Diagonal movement.** **[BLOCKS IMPLEMENTATION]** Allowed? Same cost as cardinal? This decides LoS and corner math for dungeon generation *and* the adjacency/AoO rules for combat — it's the first design decision needed before the grid, movement, or dungeon-gen code gets written. (DCSS is 8-way; Tibia is 8-way with heavily penalized diagonals.)
 4. **Origin-tile timing during a glide.** Is the departed tile freed at glide start or held until the glide ends? Affects chase/kiting feel, body-blocking in corridors, and whether allies can file through a chokepoint tightly behind each other.
 5. **Stepping away from the keyboard.** Real-time multiplayer can't pause. Since there is no action queuing (2.2.5), nothing runs away while AFK — after the current commit (seconds) finishes, the player simply stands idle. The real question is idle *exposure*: do monsters wander? Is there aggro at rest? Are there safe/rest tiles or zones where standing still is genuinely safe for a bathroom break?
 6. **Ranged combat & line-of-sight.** Entirely absent from the spec so far. Ranged kiting is the main threat to the "no perpetual kiting" pillar (Appendix: "Why lock movement too?") — a ranged build that glides, shoots, glides again needs the same hard-choice pressure melee has. Needs its own pass alongside the build system.
-

Changelog

- **v0.3 (2026-07-15)** — Named the game (Rogue's Oath). Decisions: 2D top-down tile presentation; networking plumbing sourced from Magick With Friends `framework/` (scope of reuse made explicit, continuous-sync components excluded); commit feedback extended to movement with server-authoritative verdicts (2.2.8); mid-run join moved explicitly out of scope. Added Part 4 — Open Questions.
- **v0.2** — AI-generated draft from Jon & Jeff's design conversation (PDF).